

PARCS-Agent: Enabling AI Agents to Exploit Distributed Parallel Computing via the Model Context Protocol

Oleksii Bohusevych ¹ 

¹ Faculty of Computer Science and Cybernetics, Taras Shevchenko National University of Kyiv, 01601 Kyiv, Ukraine; oleksii.bohusevych@knu.ua

* Correspondence: oleksii.bohusevych@knu.ua

1 Abstract

Large language model (LLM) agents have demonstrated impressive capabilities in tool use, code generation, and scientific reasoning, yet they remain bound by the throughput of a single execution thread. Computationally intensive tasks—Monte Carlo simulation, combinatorial optimisation, hyperparameter search, large-scale graph analysis—either time out or require severe undersampling when handled sequentially. We present PARCS-Agent, a system that exposes a cloud-scale distributed computing cluster to an AI agent through three Model Context Protocol (MCP) tools: `get_cluster_info`, `create_session`, and `run_layer`. The agent submits arbitrary C# computation logic; the platform compiles it with Roslyn at runtime, fans it out across up to 21 Kubernetes worker pods via Google Cloud Pub/Sub and KEDA autoscaling, and returns aggregated results. We introduce the PARCS-Agent Benchmark, a suite of 15 tasks spanning quantitative finance, bioinformatics, epidemiology, machine learning, cryptography, and atmospheric science, designed so that the sequential wall-clock cost substantially exceeds practical agent time budgets. Experiments show that PARCS-enabled agents achieve mean speedups of 15–18 $\times$ over sequential baselines while delivering statistically equivalent or superior solution quality. PARCS-Agent is, to our knowledge, the first system to use MCP as the interface between an AI agent and a live distributed HPC cluster, and the first benchmark to measure *speedup ratio* as a first-class agent capability metric.

Keywords: AI agents; Model Context Protocol; distributed computing; parallel computation; LLM tool use; benchmarking; Kubernetes; high-performance computing

1. Introduction

2. Related Work

2.1. Tool-Augmented Language Model Agents

Early work on augmenting language models with external tools showed that models could learn, without supervised labels, when and how to call a calculator, a search engine, or a translation API [1]. ReAct extended this idea by interleaving reasoning traces with actions such as web lookups and database queries, showing that grounding reasoning steps in real observations leads to better task completion [2]. Both papers treat tools as stateless, single-call functions that return a value and exit - a model that works well for retrieval and simple arithmetic but does not fit long-running or parallelisable computation.

Later systems scaled tool use to much larger API surfaces. ToolLLM [3] trained a model to navigate a collection of 16,000 REST endpoints, while Gorilla [4] addressed the specific problem of hallucinated API signatures in ML framework calls. SciAgent [5] moved into scientific reasoning, wrapping domain algorithms as callable tools across mathematics, physics, and finance. Mialon et al. [6] survey the broader landscape and categorise augmented language models along the dimensions

of memory, reasoning, and action; they note computation as a relatively underexplored tool type. What all these systems share is that each tool call is local, synchronous, and bound to a single process. PARCS-Agent takes a different route: `run_layer` submits user-written C# code to a Kubernetes cluster, fans it across up to 21 concurrent workers, and returns their aggregated output - the tool itself is a distributed job.

The closest prior work in terms of parallelism is LLMCompiler [7], which analyses a user query, builds a dependency graph of tool calls, and runs independent branches simultaneously. This cuts latency by up to $3.7\times$ on multi-hop question answering. The difference is that LLMCompiler parallelises *calls to existing tools* - running several web searches at once, for instance. It does not compile new code or provision workers. CodeAct [8] goes further by treating Python code as the agent's universal action, running it in a persistent interpreter that accumulates state between turns. Execution is still single-threaded and local; PARCS-Agent compiles the submitted code once and distributes the binary to remote pods, which run in parallel.

2.2. LLMs for High-Performance and Parallel Computing

The question of whether LLMs can write parallel code has received growing attention. Nichols et al. [9] benchmark code generation in MPI, OpenMP, CUDA, and Kokkos across 420 problems and find that correctness rates drop sharply compared to equivalent serial implementations, particularly as problem complexity grows. HPC-Coder-V2 [10] tries to close this gap by fine-tuning on a corpus of 120,000 synthetic parallel code samples and reports gains on MPI and OpenMP generation. Separate work evaluating frontier models on classical HPC kernels - conjugate gradient, matrix multiply, and similar - found that even strong models struggle with non-trivial memory access patterns [11].

PARCS-Agent sidesteps the parallel code generation problem entirely. The agent writes only per-worker logic as a plain C# class; the platform takes care of thread scheduling, inter-pod communication, and result collection. This means the agent never needs to reason about synchronisation or data decomposition at the code level - it just describes what a single worker should do, and the runtime handles the rest. The gap identified by ParEval is real but becomes irrelevant when the parallel infrastructure is a managed service.

He et al. [12] use LLMs to automate mapper optimisation in the Legion distributed runtime, replacing expert tuning that previously took days. This is a legitimate and useful application, but it assumes a parallelised program already exists and targets a narrow optimisation problem. PARCS-Agent is concerned with an earlier stage: an agent that has no pre-existing parallel program and wants to run a new computation in parallel from scratch.

2.3. Agent Benchmarks for Computational Tasks

Several benchmarks now measure how well agents handle complex, multi-step tasks. GAIA [13] uses 466 real-world questions that require web browsing, file reading, and multi-modal reasoning; the hardest questions remain largely unsolved. SWE-bench [14] measures whether agents can resolve real GitHub issues by producing patches that pass existing tests, with top systems now clearing around 50% of the verified subset. ScienceAgentBench [15] focuses on data-driven science, drawing 102 tasks from 44 published papers across bioinformatics, chemistry, GIS, and psychology; the best agent manages about a third of them. TheAgentCompany [16] simulates workplace tasks including software engineering and data analysis.

What none of these benchmarks test is whether an agent can exploit parallelism to get more done. Every task is designed to be completable sequentially, and wall-clock time is at most a secondary concern. The PARCS-Agent benchmark is built around the opposite premise: each of the 15 tasks requires compute that is impractical to complete in sequence within a reasonable budget. The gap between sequential and parallel agents is therefore the central result, and speedup ratio is reported alongside solution quality. Using speedup as a primary agent evaluation metric is, to the best of our knowledge, new.

2.4. The Model Context Protocol

Anthropic introduced the Model Context Protocol (MCP) in 2024 as a standard JSON-RPC-over-SSE interface for connecting LLM agents to external tools and data sources [17]. Hou et al. [18] survey the ecosystem and flag security concerns around prompt injection and tool poisoning, but find no work on using MCP to front distributed compute infrastructure. A separate study of published MCP tool descriptions found that more than half lacked adequate documentation of purpose or parameters [19]; this influenced the design of the PARCS-Agent tools, which include explicit parameter bounds and example values in their descriptions.

No prior work appears to have used MCP as the interface between an AI agent and a live HPC cluster. PARCS-Agent is the first system to do so, and the first benchmark designed to measure agent performance specifically on tasks where parallel compute access is the deciding factor.

3. System Architecture

4. PARCS-Agent Benchmark

5. Experiments

6. Discussion

7. Conclusions

Author Contributions: Conceptualization, O.B.; methodology, O.B.; software, O.B.; validation, O.B.; formal analysis, O.B.; investigation, O.B.; data curation, O.B.; writing - original draft preparation, O.B. The author has read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The benchmark dataset is publicly available at <https://huggingface.co/datasets/parcs-benchmark/parcs-agent-benchmark>. The platform source code is available at <https://github.com/alexbohuslav/parcs7>.

Acknowledgments: In this section you can acknowledge any support given which is not covered by the author contribution or funding sections. This may include administrative and technical support, or donations in kind (e.g., materials used for experiments).

Conflicts of Interest: Declare conflicts of interest or state “The authors declare no conflicts of interest.” Authors must identify and declare any personal circumstances or interest that may be perceived as inappropriately influencing the representation or interpretation of reported research results. Any role of the funders in the design of the study; in the collection, analyses or interpretation of data; in the writing of the manuscript; or in the decision to publish the results must be declared in this section. If there is no role, please state “The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results”.

Abbreviations

The following abbreviations are used in this manuscript:

MCP	Model Context Protocol
LLM	Large Language Model
HPC	High-Performance Computing
API	Application Programming Interface
KEDA	Kubernetes Event-Driven Autoscaling
GKE	Google Kubernetes Engine
SA	Simulated Annealing
VaR	Value-at-Risk
CVaR	Conditional Value-at-Risk

Appendix A

Appendix A.1

The appendix is an optional section that can contain details and data supplemental to the main text—for example, explanations of experimental details that would disrupt the flow of the main text but nonetheless remain crucial to understanding and reproducing the research shown; figures of replicates for experiments of which representative data are shown in the main text can be added here if brief, or as Supplementary Data. Mathematical proofs of results not central to the paper can be added as an appendix.

Table A1. This is a table caption.

Title 1	Title 2	Title 3
Entry 1	Data	Data
Entry 2	Data	Data

Appendix B

All appendix sections must be cited in the main text. In the appendices, Figures, Tables, etc. should be labeled, starting with “A”—e.g., Figure A1, Figure A2, etc.

References

- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*; Curran Associates: Red Hook, NY, USA, 2023.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*; OpenReview.net, 2023.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*; OpenReview.net, 2024.
- Patil, S.G.; Zhang, T.; Wang, X.; Gonzalez, J.E. Gorilla: Large Language Model Connected with Massive APIs. *arXiv* **2023**, arXiv:2305.15334.
- Ma, Z.; Gou, Z.; Hao, J.; Xu, K.; Hu, Z.; Yin, C.; Zheng, H.; Huang, M.; Deng, Y. SciAgent: Tool-Augmented Language Models for Scientific Reasoning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024.
- Mialon, G.; Dessì, R.; Lomeli, M.; Nalmpantis, C.; Pasunuru, R.; Raileanu, R.; Rozière, B.; Schick, T.; Dwivedi-Yu, J.; Celikyilmaz, A.; et al. Augmented Language Models: A Survey. *Trans. Mach. Learn. Res.* **2023**.
- Kim, S.; Moon, S.; Tabrizi, R.; Lee, N.; Mahoney, M.W.; Keutzer, K.; Gholami, A. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*; PMLR: Honolulu, HI, USA, 2024; Vol. 235, pp. 24167–24190.

8. Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; Peng, H.; Ji, H. Executable Code Actions Elicit Better LLM Agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*; PMLR: Honolulu, HI, USA, 2024; Vol. 235, pp. 50908–50933.
9. Nichols, D.; Davis, J.H.; Xie, Z.; Rajaram, A.; Bhatele, A. Can Large Language Models Write Parallel Code? In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC 2024)*; ACM: New York, NY, USA, 2024; pp. 117–130.
10. Nichols, D.; Davis, J.H.; Bhatele, A. HPC-Coder-V2: Studying Code LLMs Across Low-Resource Parallel Languages. *arXiv* **2024**, arXiv:2412.15178.
11. Xu, C.; Li, M.; Xu, X.; Qin, X.; Chen, P. LLM & HPC: Benchmarking DeepSeek’s Performance in High-Performance Computing Tasks. In *Proceedings of ISC High Performance 2025*; Springer: Cham, Switzerland, 2025.
12. He, W.; Shen, Z.; Nie, A.; Luo, C.; Nus, T.; Aiken, A.; Zaharia, M. Improving Parallel Program Performance with LLM Optimizers via Agent-System Interfaces. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*; PMLR, 2025.
13. Mialon, G.; Fourrier, C.; Swift, C.; Wolf, T.; LeCun, Y.; Scialom, T. GAIA: A Benchmark for General AI Assistants. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*; OpenReview.net, 2024.
14. Jiménez, C.E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; Narasimhan, K. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*; OpenReview.net, 2024.
15. Chen, Z.; Shi, Z.; Wang, S.; et al. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*; OpenReview.net, 2025.
16. Xu, F.F.; Ye, Y.; Jain, A.; Du, Y.; et al. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *arXiv* **2024**, arXiv:2412.14161.
17. Anthropic. Model Context Protocol. Available online: <https://modelcontextprotocol.io> (accessed on 1 May 2026).
18. Hou, X.; Zhao, Y.; Wang, S.; Wang, H. Model Context Protocol (MCP): Landscape, Security Threats and Future Research Directions. *arXiv* **2025**, arXiv:2503.23278.
19. Luo, L.; et al. Model Context Protocol (MCP) Tool Descriptions Are Smelly! *arXiv* **2025**, arXiv:2602.14878.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.